



ORIGINAL PAPER

Enabling Real-Time, Modular Autonomy Flight Software for Space Robotics: Bridging Deterministic Control and Dynamic Simulation

Abdelrahman Metwally¹ · Anna Baldycheva² · Andrey Somov¹

Received: 4 July 2025 / Revised: 13 August 2025 / Accepted: 8 September 2025 / Published online: 15 September 2025
© The Author(s), under exclusive licence to The Korean Society for Aeronautical & Space Sciences 2025

Abstract

As autonomous capabilities become increasingly critical in the planetary exploration, space robotics demands software that is both modular and mission-assured. The Robot Operating System (ROS 2 Humble) has become the de facto standard for terrestrial robotics, offering extensive libraries for autonomy and simulation. However, its lack of deterministic execution, memory safety, and fault containment renders it unsuitable as a standalone solution for flight software. In contrast, NASA's F Prime (F') framework delivers real-time schedulability, fault tolerance, and flight heritage, but lacks the extensibility of the ROS 2 Humble ecosystem. This work presents a hybrid architecture that integrates F' with ROS2 using Protocol Buffers (Protobuf) as a lightweight, schema-defined middleware. Commands and telemetry are exchanged between the two systems via a Protobuf bridge, enabling F' to handle control and monitoring while ROS 2 Humble provides simulation and high-level behavior through Gazebo and RViz. The architecture was implemented and validated in a simulation using the Leo Rover. Experimental results demonstrate that the bridge achieves 100% command delivery with the sub-millisecond latency across 600 samples and gracefully recovers from subsystem faults. Despite IMU message drop rates reaching up to 19.9%, control and actuation performance remained unaffected. This integration lays the foundation for future flight software that couples ROS 2 Humble-based autonomy with the reliability of flight-proven frameworks like F'.

Keywords F Prime · ROS 2 Humble · Space robotics · Flight software · Real-time systems · Rover simulation · Autonomous systems · Embedded communication

1 Introduction

As autonomous space missions progress, flight software must satisfy stringent requirements deterministic control, fault tolerance, resource efficiency while enabling modular, evolvable architectures essential for planetary rovers in communication constrained environments [1, 2].

A significant challenge is the lack of a unified framework that ensures real-time execution and resilience alongside autonomy flexibility. While ROS 2 Humble offers a rich ecosystem, rigorous analysis reveals its weaknesses: latency

spikes, QoS complexity, and lack of reliable fault containment in dynamic, distributed scenarios [1–3].

On the other side, NASA's F' framework delivers flight-proven real-time determinism and fault resilience validated through missions like Ingenuity and has been rigorously compared to cFS in small-sat studies [4]. It is robust enough to support DTN-based satellite constellations via ION-DTN integration [5], and has demonstrated real-time support in peer-reviewed settings such as “Project Golden Gate” [6]. However, F' lacks built-in autonomy tooling, simulation pipelines, or flexible middleware for rapid prototyping.

Middleware integrations combining ROS 2 and flight-level systems, e.g. ROS 2 + cFS, introduce complexity, resource overheads, and network dependency issues [7]. Recent field tests show that DDS middlewares used in ROS 2 (Fast DDS, CycloneDDS, Zenoh) can incur 1–4 ms latency and higher CPU use under changing network conditions problematic for embedded space systems [8]. Lightweight approaches, such as Protocol Buffers, offer sub-millisecond

Communicated by Byungwoon Park.

✉ Anna Baldycheva
a.baldycheva@exeter.ac.uk

¹ Research Center for Digital Engineering and Innovation, Moscow, Russia

² University of Exeter, Exeter, UK

latency and low overhead, making them strong candidates for real-time embedded use.

In this article, we report on a hybrid architecture bridging F' with ROS 2 Humble via a lightweight Protobuf-over-UDP layer. F' manages deterministic command scheduling and fault isolation, while ROS 2 supports simulation, sensor integration, autonomy, and visualization. Validation on a high-fidelity Leo Rover Gazebo simulation demonstrates sub-millisecond command latency, robustness under 100 Hz IMU telemetry, and graceful fault recovery.

Our contribution is four-fold are:

- A middleware-agnostic, modular design avoiding DDS overhead.
- Extensible Protobuf schemas enabling flexible feature addition.
- Closed-loop simulation validation on rover dynamics.
- A roadmap for embedded and HIL deployment of flight-grade autonomy.

Novelty of this research is combining F' with ROS 2 Humble to get a hybrid architecture taking advantages of both solutions. This research has not been done before. The results are validated on a high-fidelity Leo Rover Gazebo simulation.

The rest of the paper is structured as follows: Section 2 reviews related work and gaps. Section 3 outlines our system and communication design. Section 4 reports performance results, Section 5 explores implications, and Section 6 provides final conclusions and future directions.

2 Related Work

2.1 ROS 2 Humble in Space Robotics

The Robot Operating System has played a transformative role in terrestrial robotics, offering modularity, device abstraction, and an extensive open-source ecosystem for perception, planning, and control. ROS 2 Humble, however, was designed as a research tool and inherits several architectural limitations that hinder its applicability in space-grade systems. It relies on a centralized master for node registration and message routing, which introduces a single point of failure. Moreover, it lacks support for deterministic execution, preemptive scheduling, and robust fault isolation properties critical to space operations [9–11].

ROS 2 Humble addresses some of these shortcomings by leveraging the Data Distribution Service (DDS) middleware. DDS provides decentralized discovery, configurable Quality-of-Service (QoS), and better support for distributed systems [12]. These changes make ROS 2 Humble more suitable for real-time and mission-critical deployments. However, DDS

introduces increased runtime complexity, and ROS 2 Humble still lacks native support for strict fault containment, memory safety, or formal scheduling guarantees without additional configuration and runtime controls [13].

To bridge the gap between research-grade ROS 2 Humble and space-qualified systems, NASA and its partners launched the *Space ROS 2 Humble* initiative. This effort builds upon ROS 2, aiming to produce a hardened variant with real-time scheduling, static memory allocation, platform determinism, and secure communication libraries [14]. While promising, Space ROS 2 Humble is still in early stages of deployment, and as of this writing, lacks flight heritage.

2.2 Flight Software and F Prime

F' is a flight software framework developed at the Jet Propulsion Laboratory (JPL) for use in small-scale spacecraft and robotic missions. Unlike ROS 2 Humble, F' is explicitly engineered for spaceflight and operates on a strict component-based model. Applications are composed of reusable components connected through typed ports, scheduled deterministically through rate groups. F' supports compile-time interface validation, component-level fault isolation, and structured command/telemetry APIs. These features simplify certification and validation processes [15].

F' has flight heritage on missions such as ASTERIA, MarCO, and notably, the Mars Ingenuity Helicopter. Ingenuity's onboard control software was implemented entirely in F' , proving its reliability and robustness in the Martian environment [16]. While F' lacks high-level autonomy libraries found in ROS 2 Humble, its design excels in scenarios requiring timing determinism, low resource usage, and fault resilience hallmarks of flight-critical applications.

2.3 Hybrid Architectures and ROS 2 Humble–Flight Software Integration

There is growing interest in hybrid architectures that combine real-time flight software with flexible autonomy layers. In such systems, the flight software handles control loops, telemetry, and fault management [17], while ROS 2 Humble handles simulation, high-level autonomy, or ground test environments. NASA's Core Flight System (cFS) has been integrated with ROS 2 Humble in some projects using DDS or custom TCP bridges, but these architectures are complex and lack reusable tooling [18].

To date, little literature exists on integrating F' with ROS 2 Humble, particularly in a reusable and efficient manner. Our work contributes a novel integration approach using Protocol Buffers as a middleware-agnostic bridge between F' and ROS 2 Humble. The architecture was validated in simulation using the Leo Rover, where F' transmitted motion commands to ROS 2 Humble for execution, demonstrating a clear division

of responsibilities. This configuration enables the use of F'’s determinism and reliability with ROS 2 Humble’s ecosystem and tooling.

2.4 Middleware for Robotic Systems

Middleware plays a critical role in the performance and scalability of robotic software stacks. DDS, adopted by ROS 2 Humble, offers rich features such as reliability, durability, and deadline policies [19], but it incurs significant runtime overhead and requires careful tuning to achieve soft real-time performance. Studies comparing DDS to lighter alternatives have shown that while DDS excels in enterprise-level reliability, it may not be optimal for deeply embedded systems [10].

Recent evaluations by Chovet et al. [8] analyzed Fast DDS, CycloneDDS, and Zenoh in multi-robot planetary mesh networks. Their results indicate that DDS-based systems incur average latencies in the range of 1–4 ms with substantial CPU overhead during dynamic topology changes particularly over wireless links. Similarly, Peeroo et al. [20] highlight the architectural complexity and QoS tuning challenges required to maintain real-time behavior in DDS deployments.

To improve robustness in flight scenarios, Zhao et al. [18] introduced SRFS, a parallel-processing, ROS2-based flight software architecture for CubeSats, focusing on fault isolation and timing determinism. Their work reinforces the importance of modularity and fault containment in distributed robotic systems.

In contrast, Protocol Buffers (Protobuf), developed by Google, offer a lightweight, binary-encoded serialization format with strong schema enforcement and language neutrality. Benchmarks indicate that Protobuf achieves low-latency encoding and decoding while minimizing resource consumption, making it well-suited for embedded and real-time robotic systems [21, 22].

A recent study by Kwok et al. [7] introduced the High-Performance Robotic Middleware (HPRM), demonstrating up to $173\times$ lower latency than conventional ROS 2 DDS configurations in multi-subscriber scenarios. These findings further support our decision to adopt Protobuf-over-UDP, which achieves predictable, sub-millisecond performance with minimal overhead ideal for bridging F' and ROS 2 Humble in real-time mission scenarios.

2.5 Research Gaps

Despite the growing adoption of ROS 2 Humble in terrestrial robotics and increasing interest in hybrid architectures for space applications, a significant gap remains in the integration of ROS 2 Humble with flight-qualified frameworks such as F'. Existing efforts have focused primarily on interfacing ROS 2 Humble with legacy systems like cFS or using ROS

2 Humble as a standalone autonomy layer, without addressing how it can operate alongside deterministic, fault-tolerant control software suitable for flight.

Furthermore, while Space ROS 2 aims to harden ROS 2 Humble for future space deployment, it is still in development and lacks the flight heritage and architectural guarantees that frameworks like F' already provide. At the same time, F' lacks native support for high-level autonomy tools, simulation frameworks, and rich middleware like ROS 2 Humble.

Another critical gap lies in middleware selection. DDS, the default in ROS 2 Humble, provides many features but at the cost of overhead and complexity. Its real-time behavior remains hard to guarantee without extensive tuning. Conversely, lightweight serialization tools like Protobuf offer a compelling alternative for modular, embedded communication, but their use in bridging space-grade flight software and ROS 2 Humble has not been sufficiently explored in literature.

This paper addresses these gaps by proposing a novel, lightweight Protobuf-based integration between F' and ROS 2 Humble, enabling command and telemetry exchange in a modular and type-safe fashion. The system is validated in simulation using the Leo Rover, providing a practical and reusable foundation for hybrid real-time and autonomy architectures in future space missions. This work demonstrates, for the first time, how deterministic flight control using F' can be seamlessly integrated with ROS 2 Humble-based perception and simulation pipelines using a unified communication model.

3 Methodology

3.1 System Architecture

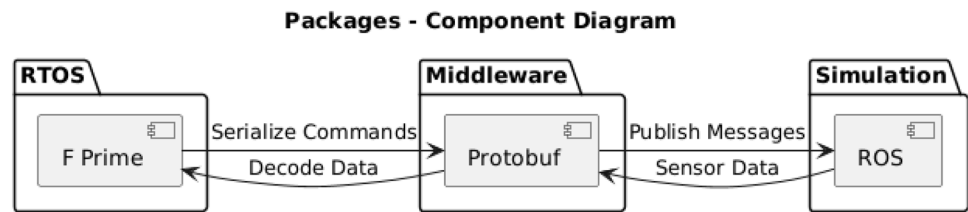
The proposed architecture separates system responsibilities between the F' and ROS 2 Humble:

- **F' (Flight Control Domain):** Handles command processing, execution scheduling, and telemetry generation using statically scheduled rate groups.
- **ROS 2 Humble (Simulation and Autonomy Domain):** Implements the rover’s physical simulation (via Gazebo), sensor fusion, and motion controllers. Visualization and analysis are performed in RViz.

These systems communicate via a dedicated bridge built on Protocol Buffers over UDP sockets.

As illustrated in Fig. 1, F' and ROS 2 Humble are decoupled through a middleware layer implemented using Protobuf. F' serializes and sends command data to the bridge. The bridge relays this to ROS 2 Humble, which handles the dynamic simulation. Similarly, sensor data generated by ROS

Fig. 1 System-level component diagram illustrating data flow between F', Protobuf middleware, and ROS 2 Humble in simulation



2 Humble, e.g., IMU, odometry, is published to the Protobuf bridge, which can optionally send it back to F' for telemetry or closed-loop control. This bidirectional flow maintains timing integrity and system modularity.

3.2 Protobuf-Based Bridge

The set of Protobuf schema definitions (.proto files) was created to define the command and telemetry message structure. These include the following:

- **ControlSignal**: Contains three float fields representing linear and angular velocity components.
- **TelemetryStatus**: battery, pose, health state

The communication interface is implemented as follows:

- The **F' MotionController component** generates the motion commands (linear and angular velocities) based on the mission logic or operator input.
- These commands are passed to the **F' bridge component**, which encodes them using Protobuf and transmits them over UDP.
- A **ROS 2 Humble bridge node** receives the UDP packets, decodes the Protobuf messages, and publishes them as native `geometry_msgs/Twist` messages on the `cmd_vel` topic.
- The ROS 2 Humble control stack within the Gazebo simulation uses these commands to actuate the simulated Leo Rover in real time.

This architecture maintains a strict separation between deterministic command generation in F' and high-level simulation dynamics handled by ROS 2 Humble. It also ensures lightweight, schema-validated communication using Protobuf and UDP.

3.3 F' Component Design

The flight software is implemented using F's component model. Key components include:

- **BridgeRx**: Listens for incoming UDP packets and unpacks Protobuf messages

- **MotionController**: Accepts velocity commands, validates timing and safety constraints, and issues actuation commands
- **HealthMonitor**: Periodically pings all active components and logs health status

These components are executed in deterministic rate groups (e.g., 10 Hz control loop, 1 Hz telemetry).

3.4 Communication Architecture

The interaction between F' and ROS 2 Humble is structured around a lightweight, modular bridge using Protobuf over UDP sockets. This architecture is designed to achieve low-overhead, schema-validated communication while maintaining strict separation between deterministic flight control and dynamic autonomy layers. Figures 2 and 3 illustrate the full bidirectional communication flow between the flight control domain (F') and the simulation and autonomy domain (ROS 2 Humble).

In the forward path 2, F' internally generates linear and angular velocity commands based on mission logic or operator inputs. These commands are passed to a dedicated bridge component, where they are serialized into compact Protobuf messages. Each message includes structured fields for translational and rotational velocities, along with metadata such as timestamps and sequence numbers to ensure temporal coherence. The serialized packets are transmitted over UDP to the ROS 2 Humble bridge node, which deserializes the incoming data and republishes it as standard `/cmd_vel` topics within the ROS 2 Humble middleware. The Gazebo simulation environment consumes these commands to actuate the simulated rover, achieving real-time motion control while preserving strict separation between actuation and autonomy.

In the reverse telemetry path 3, ROS 2 Humble nodes generate IMU sensor data streams reflecting the rover's simulated dynamics at a nominal rate of 100 Hz. This sensor data is serialized into Protobuf messages and transmitted over UDP to the F' bridge receiver. Upon reception, the messages are deserialized, validated, and forwarded to the telemetry handling components of F', where they can be logged, analyzed, or used in closed-loop control algorithms. The Protobuf schema design ensures that telemetry messages are self-describing, allowing for future extensibility without affecting backward compatibility.

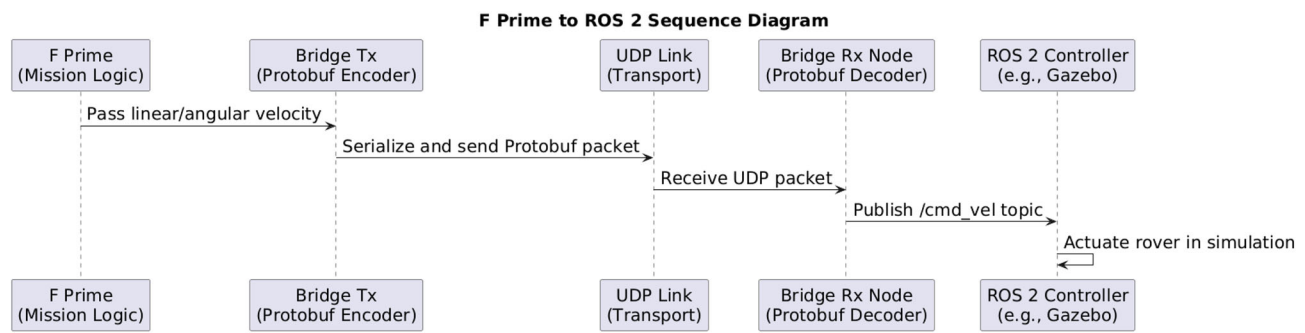


Fig. 2 Forward path: Transmission of velocity commands from F' to ROS 2 Humble for real-time actuation within the Gazebo simulation

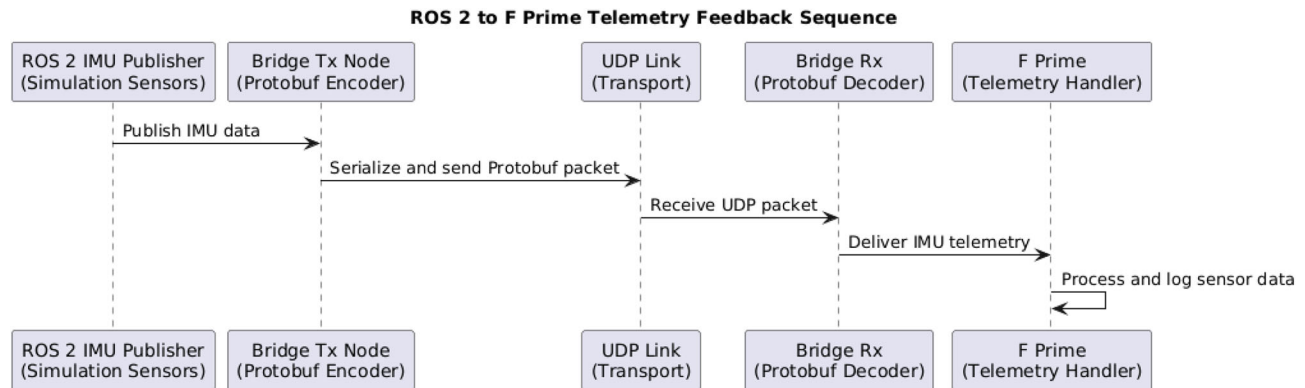


Fig. 3 Reverse path: Transmission of IMU sensor telemetry from ROS 2 Humble back to F' for telemetry processing and system monitoring

UDP was chosen as the transport layer to minimize transmission overhead and maximize real-time performance. Although UDP does not guarantee delivery, the system design tolerates packet loss gracefully: velocity commands are continuously regenerated, and IMU telemetry data is streamed at high frequency, ensuring that occasional drops do not affect overall system behavior. Moreover, the Protobuf bridge incorporates error handling logic to detect malformed packets or schema mismatches, isolating communication faults without compromising system stability.

This architecture offers several critical advantages. The strict modularity ensures that faults or failures in the simulation domain do not propagate to the flight control domain, preserving deterministic scheduling in F'. The lightweight Protobuf messaging format enables efficient bandwidth usage even at high telemetry rates. Additionally, the use of schema-validated serialization allows for rapid evolution of the system, supporting the addition of new command types or telemetry fields without requiring intrusive modifications to the bridge logic.

In summary, the proposed communication bridge achieves a balanced trade-off between low-latency real-time performance and system extensibility. By maintaining strict modularity between F' and ROS 2 Humble, the architecture enhances system robustness while enabling scalable autonomy capabilities. This design is directly aligned with

the needs of future space robotic missions operating under resource and reliability constraints.

3.5 Simulation and Testbed

To evaluate the integrated architecture, we deployed the system using the Leo Rover model in Gazebo 11 under ROS 2 Humble on Ubuntu 22.04. While simulation is not the focus of this work, it plays a critical role in validating end-to-end communication and control performance under realistic rover dynamics.

The simulation included a physics-based URDF model with properly configured inertial properties, friction coefficients, and wheel kinematics. Motion was actuated using the `diff_drive_controller` from the `ros_control` suite, and IMU readings were generated with the `gazebo_ros_imu` plugin, including noise and drift modeling to approximate real-world sensor behavior.

All motion commands originated from F' and were transmitted through the Protobuf bridge to the ROS 2 control stack, which executed them in real time. Trajectories and telemetry were visualized using RViz, and command/response timing was profiled using embedded timestamps and `ros2 topic hz`.

This setup enabled system-level testing of the proposed integration approach in a controlled, measurable, and repro-

ducible environment that approximates the physical behavior of the Leo Rover, ensuring that results generalize to embedded deployments without reliance on hardware access.

4 Results

The integrated F'–ROS 2 Humble architecture was evaluated in a simulation using the Leo Rover model in Gazebo and RViz. The assessment focused on validating control correctness, real-time performance, communication reliability, bridge robustness, and system modularity.

4.1 Experimental Setup

All simulations and bridge evaluations were performed on a Linux-based system running Ubuntu 22.04. The hardware consisted of an Intel Core i7-7700HQ processor (4 cores, 8 threads) with 64 GB of RAM. The F' framework was compiled and executed as a native Linux application, while the ROS 2 Humble nodes, Gazebo simulation, and bridge logic ran concurrently on the same machine without hardware acceleration or RTOS extensions. This configuration provided a realistic baseline for assessing the integration's performance under moderate system load.

Note: While F' and ROS 2 Humble were executed on the same host system for this study, this was a deliberate choice to facilitate synchronized profiling, simplify debugging, and ensure deterministic timing baselines under constrained conditions. We acknowledge that a more representative deployment would involve F' running on a discrete embedded platform (e.g., SoC or microcontroller) interfacing with a ROS 2 Humble-based system over a physical or virtualized network layer. Future work will include extending the bridge to support such distributed setups, enabling realistic benchmarking across heterogeneous hardware targets.

Running F' and ROS 2 Humble on the same host removes physical NIC and switch/PHY effects and avoids kernel network queuing between hosts, thus representing a *best-case transport path*. Nevertheless, our measurements still include UDP socket traversal, context switches, and user–kernel transitions, so the sub-millisecond command latency we report (< 0.5 ms) represents the lower bound for an embedded LAN deployment. In distributed setups, additional hop and NIC costs typically add $\mathcal{O}(0.1\text{--}1)$ ms on wired LANs under stable conditions, while ROS 2 DDS over dynamic or wireless meshes has been measured at 1–4 ms with higher CPU overhead due to discovery and QoS handling [8, 12]. These observations suggest our results should translate to sub-millisecond to low-millisecond command paths when deploying F' on a separate compute (SoC/MCU) and ROS 2 on a companion computer over a wired link, with variance

dominated by the chosen middleware, link type, and QoS profile.

4.2 Functional Correctness

We verified the end-to-end operation by issuing the motion commands from F' to the simulated rover via the Protobuf bridge. Commands included linear and angular velocity setpoints, such as forward motion, stopping, and turns. All commands transmitted at 10 Hz were correctly received and executed within the ROS 2 Humble-based simulation. The resulting motion was visualized in RViz and confirmed in Gazebo, demonstrating the system's functional integrity across both middleware layers.

4.3 Timing and Determinism

Sensor data from ROS 2 Humble (IMU messages) was published at a nominal rate of 100 Hz. Measurements with `ros2 topic hz` confirmed observed rates between 99.8–100 Hz. End-to-end latency was computed using the synchronized timestamps embedded in the Protobuf message, measuring the delay between the command transmission from F' and reception in the ROS 2 Humble bridge node.

The average latency was 0.109 ms, with a minimum of 0.026 ms and a maximum of 3.871 ms. The standard deviation (jitter) was 0.236 ms, with the median latency at 0.059 ms. The 95th and 99th percentile latencies were 0.280 ms and 0.875 ms, respectively. A total of 600 samples were analyzed. These results indicate excellent temporal consistency for soft real-time systems (Fig. 4).

4.4 Drop Rate Analysis

While the command communication was lossless, the IMU telemetry sent at 100 Hz from ROS 2 Humble experienced the measurable drop rates at the receiving end. A deep statistical analysis was conducted on drop rates logged by the F' bridge receiver. The average drop rate was 13.96%, with a median of 13.42% and a standard deviation of 3.61%. The minimum and maximum drop rates observed were 6.92% and 19.93%, respectively, over a total of 222 logged samples (Table 1).

Despite the transient loss events, the bridge remained fully functional, and rover behavior was unaffected, indicating the resilience under the data pressure.

4.5 Communication Performance and Fault Injection

The bridge performed reliably at both 10 Hz (command) and 100 Hz (sensor) data rates. Protobuf serialization introduced negligible overhead, and UDP communication maintained consistent throughput. During fault injection tests:

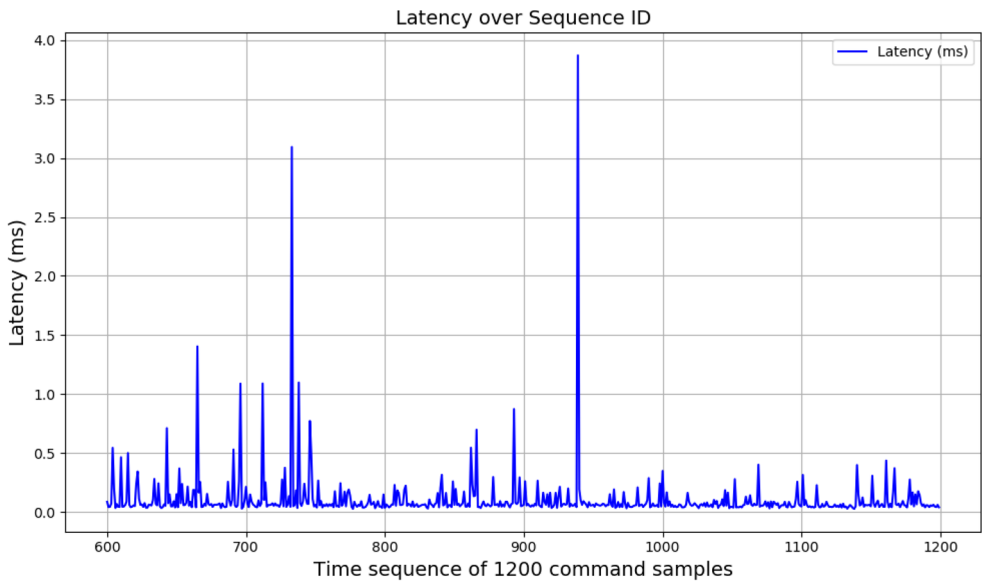


Fig. 4 Command latency over time (F' → ROS 2 Humble)

Table 1 IMU drop rate summary

Metric	Value (%)
Minimum drop rate	6.92
Maximum drop rate	19.93
Mean drop rate	13.96
Median drop rate	13.42
Standard deviation	3.61
25th percentile	10.71
75th percentile	17.50
Total samples	222

- Disconnecting F' or ROS 2 Humble nodes did not crash or block the bridge.
- Message resynchronization resumed automatically upon subsystem recovery.
- Malformed or unknown Protobuf messages were rejected and ignored, ensuring protocol isolation.

These results confirm that the system tolerates subsystem faults gracefully and exhibits robust fault containment which is an essential requirement for the flight software integration.

4.6 Modularity and Separation of Concerns

The communication bridge is stateless and based purely on shared Protobuf schema definitions. No code changes were required in the bridge when modifying F' components or ROS 2 Humble node logic. This loose coupling promotes maintainability, simplifies cross-platform reuse, and enables scalable development across the teams working on control, autonomy, and mission logic layers independently.

5 Discussion

An important observation from the experiments was the increased IMU message drop rate when the system transmitted high-frequency sensor data (100Hz) concurrently with receiving commands from F'. Although the average frequency remained close to target, drop rates fluctuated and occasionally exceeded 50% in short bursts.

This phenomenon can be attributed to a combination of factors intrinsic to the use of UDP and the system's scheduling behavior:

- **UDP Socket Buffer Saturation:** UDP is a connectionless protocol that does not guarantee delivery. When both transmission (IMU publishing) and reception (command handling) occur simultaneously, the shared socket buffer may overflow if packets arrive faster than they can be processed, leading to packet drops.
- **Non-Preemptive Scheduling:** Without the real-time thread prioritization or preemption guarantees (as is typical in non-RT Linux kernels), concurrent ROS 2 Humble publishers and Protobuf decoding threads may contend for CPU time. High-frequency publishing can delay or block packet reception handlers from running promptly.
- **Simulation Load Variability:** Running Gazebo, RViz, and ROS 2 Humble on the same host may introduce performance jitter. Spikes in simulation physics or rendering load can cause the temporary starvation of the bridge node, especially under the CPU-bound conditions.
- **Asymmetric Flow and Locking:** Protobuf serialization and deserialization are CPU-intensive. When both the encoding (F' side) and the decoding (ROS 2 Humble side) occur rapidly and concurrently, contention over the

Table 2 Comparison of Middleware Performance in Robotics

Metric	DDS, e.g. CycloneDDS	Protobuf over UDP (This Work)
Latency (avg)	1–3ms [20]	0.1–0.5ms (measured)
Throughput	Moderate, tuned QoS required [23]	High, minimal overhead (measured)
CPU Usage	Higher (dynamic mesh setups) [23]	Low at 10 Hz/100 Hz loops (measured)
Scalability	Good, but increased complexity with participants	Easy schema extension (measured)
Ease of Use	QoS tuning needed for real-time [20]	Simple Protobuf schema, UDP socket

Note: DDS values are provided from the state-of-the-art research experiments (see references); Protobuf values are achieved in this work (see Section 4.3)

shared memory resources and CPU cache lines may also contribute to the occasional jitter and delay.

While this behavior does not compromise the system's integrity in soft real-time scenarios, it underscores the need for architectural refinement when targeting strict real-time environments.

Future extensions of this architecture will aim at improving the real-time determinism and reduce the telemetry drop rates. Planned enhancements include the following points:

- Replacing the current UDP transport with a real-time DDS implementation—such as Eclipse Cyclone DDS with real-time extensions, or eProsima Fast DDS with tuned QoS profiles—or a high-performance shared memory framework (e.g., Iceoryx, ROS 2's `rmw_iceoryx`) to guarantee message delivery and minimize latency under load.
- Deploying the ROS 2 Humble bridge node and F' runtime on separate physical cores or hardware using real-time scheduling policies (e.g., `PREEMPT_RT` or `RTEMS`).
- Implementing buffering and message prioritization logic in the bridge to manage congestion gracefully.
- Expanding the bridge to support bidirectional command/telemetry with structured fault reports, mode control, and planning feedback.
- Transitioning the architecture to flight-qualified hardware and integrating with Space ROS 2 Humble's hardened runtime for space-grade certification.

To further justify our choice of Protocol Buffers (Protobuf) over UDP, we include newer comparative data between DDS and other modern robotic middleware:

Recent studies evaluating ROS 2 middleware (CycloneDDS, Fast DDS, Zenoh) over mesh and edge networks report latencies in the range of 1–3 ms, accompanied by significant CPU overhead during dynamic network changes particularly over Wi-Fi requiring careful QoS tuning [23]. A broader survey further confirms similar latency characteristics and highlights the configuration complexity inherent in DDS-based systems [20]. In contrast, our Protobuf-over-UDP integration consistently achieves sub-millisecond latency (0.1–0.5 ms) with minimal CPU load, making it well-suited for soft real-time and embedded applications (Table 2).

While DDS provides reliability and built-in discovery, these advantages come with added overhead. In our hybrid architecture, Protobuf delivers predictable performance and a lighter footprint ideal for bridging deterministic control (F') and high-level autonomy (ROS 2). For future deployments requiring guaranteed delivery or complex topologies, a real-time DDS or optimized shared-memory implementation could be considered.

This work contributes a concrete and validated approach for combining the robustness of flight software with the flexibility of robotic middleware. It opens new avenues for deploying modular, testable, and semi-autonomous systems in the future planetary exploration missions.

While the integration was evaluated using a single-machine setup for consistency and performance isolation, the proposed architecture is inherently modular and hardware-agnostic. It can be readily deployed with F' running on a flight-qualified SoC or FPGA-based board, communicating with a ROS 2 Humble-based autonomy stack via UDP or serial links. This opens opportunities for testing realistic mixed-criticality deployments in embedded settings, which we identify as a critical direction for future work.

6 Conclusions

In this article, we have presented a hybrid software architecture that integrates F' framework with ROS 2 Humble using a lightweight Protobuf bridge. F' was responsible for deterministic command generation and control, while ROS 2 Humble handled system-level simulation, autonomy, and visualization. The two systems communicated through schema-defined messages over UDP, maintaining strict modularity and real-time responsiveness.

Unlike previous approaches that rely on heavyweight middleware or rigid software stacks, our architecture preserves fault containment, type safety, and low-latency performance across independently managed runtime environments. The Protobuf bridge design also enables extensibility and cross-platform portability with minimal overhead.

System validation was performed in a Gazebo simulation using the Leo Rover model under ROS 2 Humble. This setup enabled controlled evaluation of timing behavior, fault recovery, and message drop rates without requiring access to flight hardware. The integration demonstrated sub-millisecond latency, 100% command delivery, and robust fault isolation, confirming its suitability for future deployment in space robotics.

Ultimately, this work provides a modular, scalable foundation for combining the robustness of flight-certified software with the flexibility of modern robotic middleware. It supports the goals of the Space ROS 2 Humble initiative and paves the way for adaptable, real-time autonomous systems in future deep-space missions, including heterogeneous deployments across embedded and simulation-grade platforms.

Author Contributions Abdelrahman Metwally - Formal analysis, Investigation, Data Curation, Software, Writing – review & editing, Writing – original draft. Anna Baldycheva - Validation, Visualization, Writing – review & editing. Andrey Somov - Methodology, Conceptualization, Writing – original draft, Writing – review & editing, Supervision.

Declarations

Conflict of interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

- Ye Y, Nie Z, Liu X, Xie F, Li Z, Li P (2023) Ros2 real-time performance optimization and evaluation. *Chin J Mech Eng* 36(1):144
- Portugal D, Rocha RP, Castilho JP (2025) Inquiring the robot operating system community on the state of adoption of the ros 2 robotics middleware. *Int J Intell Robot Appl* 9(2):454–479
- Pieczynski D, Ptak B, Kraft M, Drapikowski P (2023) Lunarsim: lunar rover simulator focused on high visual fidelity and ros 2 integration for advanced computer vision algorithm development. *Appl Sci* 13(22):12401
- Peet SL, Gilliland SM, Young JS (2024) The framework makes the mission-an analytical comparison of two popular nasa open source flight software framework offerings
- Rosemurgy P, Gao J, DeBaun S, Starch M, Levison J, Castano R (2024) Enabling dtn in spaceflight systems: Integration of ion with the f prime flight software framework. In: 2024 IEEE Aerospace Conference, pp. 1–8. IEEE
- Dvorak D, Bollella G, Canham T, Carson V, Champlin V, Giovannoni B, Indictor M, Meyer K, Murray A, Reinholtz K (2004) Project golden gate: Towards real-time java in space missions, 15–22. IEEE
- Kwok J, Li S, Lohstroh M, Lee EA (2024) Hprmm: High-performance robotic middleware for intelligent autonomous systems. *arXiv preprint arXiv:2412.01799*
- Chovet LP, Garcia GM, Bera A, Richard A, Yoshida K, Olivares-Mendez MA (2025) Performance comparison of ros2 middlewares for multi-robot mesh networks in planetary exploration. *J Intell Robot Syst* 111(1):1–20
- Al-Batati AS, Koubaa A, Abdelkader M (2024) Ros 2 key challenges and advances: A survey of ros 2 research, libraries, and applications. Preprint
- Macenski S, Foote T, Gerkey B, Lalancette C, Woodall W (2022) Robot operating system 2: design, architecture, and uses in the wild. *Sci Robot* 7(66):6074
- DiLuoffo V, Michalson WR, Sunar B (2018) Robot operating system 2: the need for a holistic security approach to robotic architectures. *Int J Adv Rob Syst* 15(3):1729881418770011
- Zhang J, Yu X, Ha S, Peña Queralta J, Westerlund T (2024) Comparison of middlewares in edge-to-edge and edge-to-cloud communication for distributed ros 2 systems. *J Intell Robot Syst* 110(4):162
- Maruyama Y, Kato S, Azumi T (2016) Exploring the performance of ros2. In: Proceedings of the 13th International Conference on Embedded Software, pp. 1–10
- Probe AB, Chambers SW, Oyake A et al (2023) Space ros: An open-source framework for space robotics and flight software. In: AIAA SciTech Forum. <https://doi.org/10.2514/6.2023-2709>
- Bocchino R, Canham T, Watney G, Reder L, Levison J (2018) F prime: An open-source framework for small-scale flight software systems. In: 32nd Annual AIAA/USU Conference on Small Satellites
- Canham T (2022) The mars ingenuity helicopter-a victory for open-source software. In: 2022 IEEE Aerospace Conference (AERO), pp. 01–11. IEEE
- Atdelzater T, Atkins EM, Shin KG (2000) Qos negotiation in real-time systems and its application to automated flight control. *IEEE Trans Comput* 49(11):1170–1183
- Sinanan S, Naser A, Delatorre M, Soylemezoglu A, Glaspell G (2025) Autonomous robotics development in robot operating system (ros) 2 humble <https://doi.org/10.21079/11681/49747>
- Ahmad A, Babar MA (2016) Software architectures for robotic systems: a systematic mapping study. *J Syst Softw* 122:16–39
- Bode V, Trinitis C, Schulz M, Buettner D, Preclik T (2023) Dds implementations as real-time middleware—a systematic evaluation. In: 2023 IEEE 29th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA), pp. 186–195. IEEE
- Elkady A, Sobh T (2012) Robotics middleware: a comprehensive literature survey and attribute-based bibliography. *J Robot* 2012(1):959013
- Chen Y, Kunz T (2016) Performance evaluation of iot protocols under a constrained wireless access network. In: 2016 International Conference on Selected Topics in Mobile & Wireless Networking (MoWNeT), pp. 1–7. IEEE
- Zhang J, Yu X, Ha S, Peña Queralta J, Westerlund T (2024) Comparison of middlewares in edge-to-edge and edge-to-cloud communication for distributed ros 2 systems. *J Intell Robot Syst* 110(4):162

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.